

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import geopandas as gpd
from census import Census
```

Census Data Cleaning

Request Census Data

```
In [2]: c = Census("c351645b7d72513154c6b11195f60f5493134e2b", year=2022)
data = c.acs5.get(
    (
        "NAME",
        "B01003_001E", # Total Population
        "B19013_001E", # Median Household Income
        "B17001_001E", # Povert Status
        "B17001_002E", # Below poverty
        "B23025_003E", # Civilian Labor force
        "B23025_005E", # Unemployed
        "B25003_001E", # Total Housing
        "B25044_003E", # Renter-occupied units with no vehicle available
        "B08301_001E", # Total commuters
        "B08301_010E" # Commuting by public transportation
    ),
    {"for": "tract:*", "in": "state:36 county:*"},
)
print('done')
```

done

Comfirm Census Data

```
In [3]: ny_df = pd.DataFrame(data)
print(ny_df.sample(5))
```

	NAME	B01003_001E	\			
995	Census Tract 138.02; Erie County; New York	5534.0				
5250	Census Tract 63.02; Westchester County; New York	2783.0				
2945	Census Tract 246; Niagara County; New York	4700.0				
1826	Census Tract 768; Kings County; New York	3992.0				
272	Census Tract 235.01; Bronx County; New York	3294.0				

	B19013_001E	B17001_001E	B17001_002E	B23025_003E	B23025_005E	\	
995	85833.0	5489.0	165.0	3057.0	261.0		
5250	71780.0	2644.0	505.0	1428.0	44.0		
2945	68564.0	4632.0	439.0	2555.0	55.0		
1826	71250.0	3884.0	619.0	1764.0	93.0		
272	39419.0	3294.0	1089.0	1572.0	146.0		

	B25003_001E	B25044_003E	B08301_001E	B08301_010E	state	county	tract
995	2230.0	109.0	2708.0	0.0	36	029	013802
5250	1263.0	0.0	1349.0	460.0	36	119	006302
2945	2008.0	89.0	2490.0	7.0	36	063	024600
1826	1277.0	47.0	1627.0	500.0	36	047	076800
272	1236.0	13.0	1418.0	950.0	36	005	023501

Rename and Recode Variables

```
In [4]: # provide readable names and do any needed calculations
ny_df["total_population"] = ny_df["B01003_001E"]
ny_df["median_income"] = ny_df["B19013_001E"]
ny_df["poverty_rate"] = ny_df["B17001_002E"] / ny_df["B17001_001E"] * 100
ny_df["unemployment_rate"] = ny_df["B23025_005E"] / ny_df["B23025_003E"] * 100
ny_df["no_vehicle_rate"] = ny_df["B25044_003E"] / ny_df["B25003_001E"] * 100

# Confirm recoded columns
print(ny_df.sample(5))
```

	NAME	B01003_001E	\
768	Census Tract 1500.04; Dutchess County; New York	3029.0	
1545	Census Tract 371; Kings County; New York	4935.0	
4295	Census Tract 291.06; Richmond County; New York	3439.0	
394	Census Tract 391; Bronx County; New York	7187.0	
2591	Census Tract 20; New York County; New York	4689.0	

	B19013_001E	B17001_001E	B17001_002E	B23025_003E	B23025_005E	\
768	112457.0	3016.0	282.0	1585.0	29.0	
1545	98281.0	4935.0	596.0	2762.0	169.0	
4295	118211.0	3439.0	307.0	1645.0	159.0	
394	31769.0	7181.0	2213.0	3729.0	216.0	
2591	26031.0	4689.0	1785.0	1406.0	198.0	

	B25003_001E	B25044_003E	B08301_001E	B08301_010E	state	county	tract	\
768	1085.0	70.0	1505.0	27.0	36	027	150004	
1545	1660.0	190.0	2530.0	1522.0	36	047	037100	
4295	1040.0	52.0	1482.0	164.0	36	085	029106	
394	2896.0	13.0	3457.0	1630.0	36	005	039100	
2591	2021.0	24.0	1175.0	536.0	36	061	002000	

	total_population	median_income	poverty_rate	unemployment_rate	\
768	3029.0	112457.0	9.350133	1.829653	
1545	4935.0	98281.0	12.077001	6.118755	
4295	3439.0	118211.0	8.927014	9.665653	
394	7187.0	31769.0	30.817435	5.792438	
2591	4689.0	26031.0	38.067818	14.082504	

	no_vehicle_rate
768	6.451613
1545	11.445783
4295	5.000000
394	0.448895
2591	1.187531

```
In [5]: ny_df["state"] = ny_df["state"].astype(str).str.zfill(2)
ny_df["county"] = ny_df["county"].astype(str).str.zfill(3)
ny_df["tract"] = ny_df["tract"].astype(str).str.zfill(6)

# Concatenate to form full 11-digit GEOID
ny_df["GEOID"] = ny_df["state"] + ny_df["county"] + ny_df["tract"]
```

```
In [6]: print(ny_df.sample(5))
```

	NAME	B01003_001E	\
1166	Census Tract 112; Herkimer County; New York	5829.0	
5311	Census Tract 109.02; Westchester County; New York	6241.0	
4163	Census Tract 523.05; Rensselaer County; New York	2584.0	
2280	Census Tract 722; Montgomery County; New York	3588.0	
5076	Census Tract 9550.01; Ulster County; New York	2245.0	

	B19013_001E	B17001_001E	B17001_002E	B23025_003E	B23025_005E	\
1166	67238.0	5294.0	447.0	3045.0	109.0	
5311	151921.0	6184.0	218.0	3886.0	255.0	
4163	82368.0	2576.0	124.0	1539.0	52.0	
2280	73558.0	3560.0	254.0	1712.0	32.0	
5076	68266.0	2217.0	458.0	1210.0	0.0	

	B25003_001E	B25044_003E	B08301_001E	B08301_010E	state	county	tract	\
1166	2404.0	86.0	2916.0	1.0	36	043	011200	
5311	2253.0	36.0	3577.0	344.0	36	119	010902	
4163	1100.0	0.0	1445.0	0.0	36	083	052305	
2280	1142.0	31.0	1660.0	14.0	36	057	072200	
5076	987.0	0.0	1200.0	150.0	36	111	955001	

	total_population	median_income	poverty_rate	unemployment_rate	\
1166	5829.0	67238.0	8.443521	3.579639	
5311	6241.0	151921.0	3.525226	6.562017	
4163	2584.0	82368.0	4.813665	3.378817	
2280	3588.0	73558.0	7.134831	1.869159	
5076	2245.0	68266.0	20.658548	0.000000	

	no_vehicle_rate	GEOID
1166	3.577371	36043011200
5311	1.597870	36119010902
4163	0.000000	36083052305
2280	2.714536	36057072200
5076	0.000000	36111955001

Open Street Map POIS & classification

```
In [7]: nyOSM= r'C:/Users/ALIENWARE/Desktop/Machine Learning Final/new-york-latest-free.shp
gdf_nyOSM = gpd.read_file(nyOSM)
gdf_nyOSM["fclass"].unique()
```

```
Out[7]: array(['camera_surveillance', 'bench', 'park', 'guesthouse', 'attraction',
              'school', 'viewpoint', 'beverages', 'beauty_shop', 'tourist_info',
              'museum', 'bar', 'lighthouse', 'dentist', 'police', 'restaurant',
              'fire_station', 'memorial', 'tower', 'market_place', 'ruins',
              'hotel', 'toilet', 'fast_food', 'convenience', 'pub', 'pharmacy',
              'bicycle_shop', 'clothes', 'cafe', 'water_tower', 'drinking_water',
              'shelter', 'bank', 'doityourself', 'kindergarten', 'laundry',
              'furniture_shop', 'jeweller', 'theatre', 'hairstylist',
              'garden_centre', 'sports_shop', 'doctors', 'supermarket', 'clinic',
              'post_office', 'recycling_glass', 'car_dealership', 'library',
              'college', 'graveyard', 'community_centre', 'pitch', 'university',
              'monument', 'playground', 'sports_centre', 'picnic_site', 'prison',
              'hospital', 'theme_park', 'battlefield', 'comms_tower',
              'observation_tower', 'post_box', 'artwork', 'town_hall',
              'fountain', 'stadium', 'gift_shop', 'public_building', 'atm',
              'arts_centre', 'cinema', 'embassy', 'courthouse', 'fort',
              'bookshop', 'mall', 'nursing_home', 'hostel', 'water_works',
              'stationery', 'department_store', 'car_wash', 'zoo', 'car_rental',
              'camp_site', 'waste_basket', 'butcher', 'greengrocer',
              'bicycle_rental', 'veterinary', 'mobile_phone_shop', 'bakery',
              'telephone', 'recycling_paper', 'recycling', 'general',
              'wayside_cross', 'chemist', 'archaeological', 'nightclub',
              'food_court', 'water_mill', 'motel', 'shoe_shop', 'outdoor_shop',
              'optician', 'florist', 'toy_shop', 'newsagent', 'wayside_shrine',
              'computer_shop', 'vending_machine', 'vending_any', 'car_sharing',
              'swimming_pool', 'water_well', 'ice_rink', 'caravan_site', 'kiosk',
              'biertgarten', 'chalet', 'recycling_metal', 'windmill',
              'golf_course', 'castle', 'hunting_stand', 'travel_agent',
              'video_shop', 'vending_parking', 'dog_park', 'alpine_hut',
              'recycling_clothes', 'track', 'wastewater_plant'], dtype=object)
```

```
In [8]: food_fclass = ['restaurant', 'cafe', 'fast_food', 'bakery', 'butcher', 'greengrocer',
                      'pub', 'bar', 'food_court']
```

```
# Filter the GeoDataFrame to include only those POIs
pois_food = gdf_nyOSM[gdf_nyOSM["fclass"].isin(food_fclass)].copy()

# Check how many and what types
print(pois_food["fclass"].value_counts())
print(f"Total filtered POIs: {len(pois_food)}")
```

```
fclass
restaurant    11684
fast_food     5783
cafe          3202
bar           1808
bakery         891
pub           586
greengrocer   211
butcher       191
food_court     70
Name: count, dtype: int64
Total filtered POIs: 24426
```

```
In [9]: pois_food[["name", "fclass"]].dropna().head(10)
```

Out[9]:

	name	fclass
14	Seebers Tavern	bar
19	Griffon Gastropub	restaurant
20	Tony Rome's "The Globe"	restaurant
21	Rick's on Main	restaurant
22	East End Tap Room	restaurant
23	Leo's Pizzeria	restaurant
62	Jim's Steak Out	fast_food
64	Allen Street Hardware	restaurant
65	Crust Pizza Bar	restaurant
66	Falley Allen	restaurant

```
In [10]: name_counts = pois_food["name"].value_counts()
chain_brand = name_counts[name_counts>1].index.tolist()
pois_food["biz_type"] = pois_food["name"].apply(
    lambda x: "chain" if x in chain_brand else "independent"
)
print(pois_food["biz_type"].value_counts())
```

```
biz_type
independent    17189
chain           7237
Name: count, dtype: int64
```

```
In [11]: pois_food[["name", "fclass", "biz_type"]].sample(5)
```

Out[11]:

	name	fclass	biz_type
21822	La Mision	restaurant	independent
28490	Cranberry Cafe	fast_food	independent
14554	Go Believe Bakery	fast_food	independent
49104	Halal Grill Eats	fast_food	independent
79105	Inaka Japanese Fusion	restaurant	independent

GeoMapping of NYC

```
In [12]: import contextily as cx
```

```
In [13]: nyct2020 = r'C:/Users/ALIENWARE/Desktop/Machine Learning Final/nyct2020_25a/nyct2020
gdf_nyct2020 = gpd.read_file(nyct2020)
print(gdf_nyct2020.columns)
```

```
Index(['CTLabel', 'BoroCode', 'BoroName', 'CT2020', 'BoroCT2020', 'CDEligibil',
      'NTAName', 'NTA2020', 'CDTA2020', 'CDTANAME', 'GEOID', 'PUMA',
      'Shape_Leng', 'Shape_Area', 'geometry'],
      dtype='object')
```

```
In [14]: gdf_nyct2020["BoroCode"] = gdf_nyct2020["BoroCode"].astype(int)
```

```
In [15]: nyct_ny = gdf_nyct2020[gdf_nyct2020["BoroCode"].isin([1,2,3,4,5])].copy()
```

```
In [16]: print(len(nyct_ny))
```

2325

```
In [17]: pois_food = pois_food.to_crs(nyct_ny.crs)
        pois_nyc = gpd.sjoin(pois_food, nyct_ny, predicate="within", how="inner")

        fig, ax = plt.subplots(figsize=(15, 15))

        nyct_ny.plot(ax=ax, edgecolor="white", facecolor="none", linewidth=0.1)

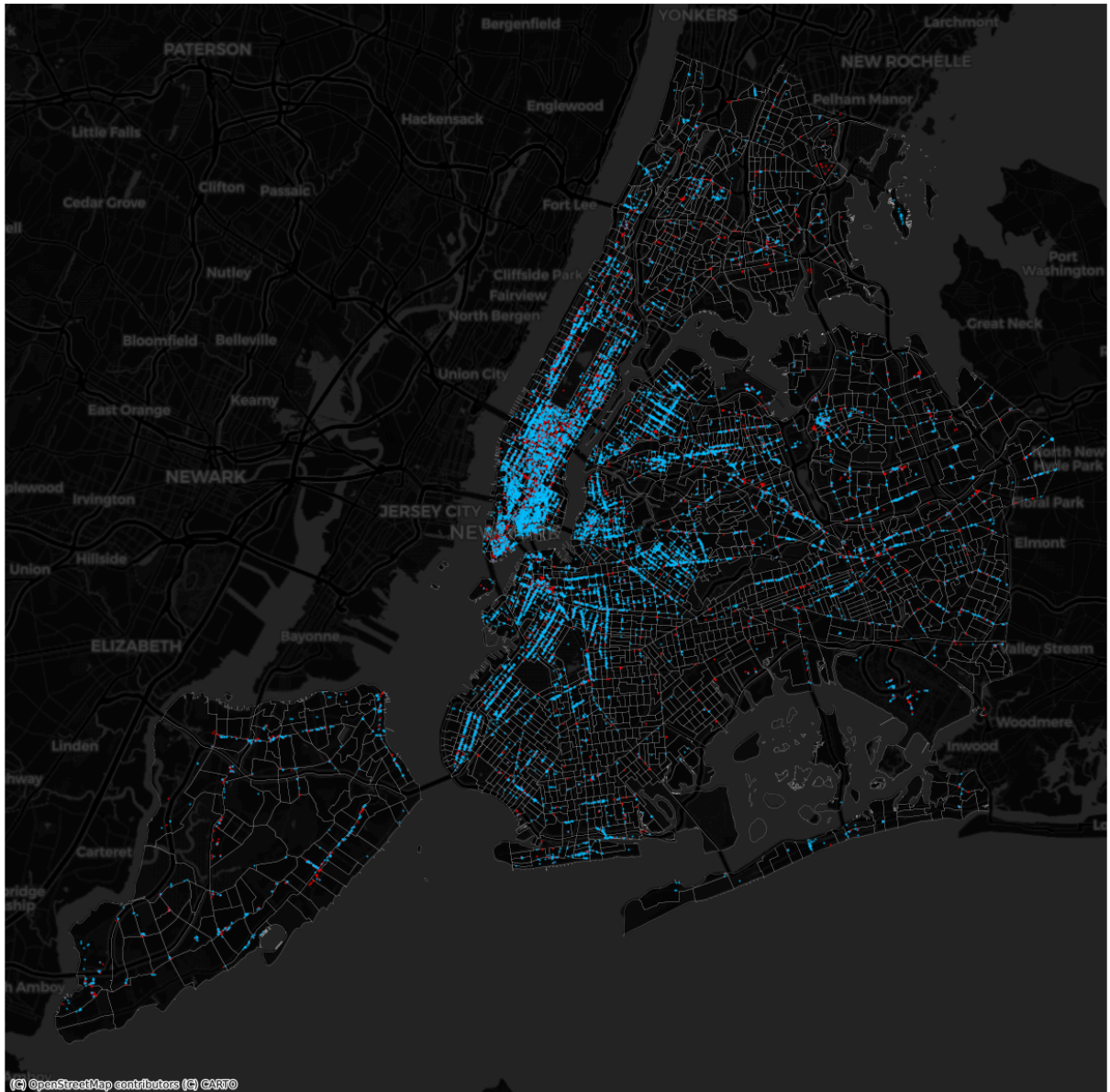
        pois_nyc[pois_nyc["biz_type"] == "chain"].plot(
            ax=ax, color="red", markersize=0.5, label="Chain", alpha=0.6)

        pois_nyc[pois_nyc["biz_type"] == "independent"].plot(
            ax=ax, color="deepskyblue", markersize=0.5, label="Independent", alpha=0.6)

        cx.add_basemap(ax, source=cx.providers.CartoDB.DarkMatter, crs=nyct_ny.crs.to_string())

        plt.title("Food-Related POIs in NYC", fontsize=14)
        plt.axis("off")
        plt.show()
```

Food-Related POIs in NYC



Chain & Independent food store hotspot in Brooklyn

```
In [18]: nyct_ny = nyct_ny.to_crs(epsg=2263)

# brooklyn = nyct_ny[nyct_ny["BoroCode"] == 3].copy()
nyc = gdf_nyct2020[gdf_nyct2020["BoroCode"].isin([1,2,3,4,5])].copy()

# Filter and project data
pois_chain = pois_food[pois_food["biz_type"] == "chain"]
pois_indie = pois_food[pois_food["biz_type"] == "independent"]

nyc_proj = nyc.to_crs(epsg=3857)
pois_indie_proj = pois_indie.to_crs(epsg=3857)

pois_indie_ny = gpd.sjoin(pois_indie_proj, nyc_proj, predicate="within", how="inner")

# Hexbin: Chain Stores
x = pois_indie_ny.geometry.x
y = pois_indie_ny.geometry.y
```



```

fig, ax = plt.subplots(figsize=(12, 12))
hb = ax.hexbin(x, y, gridsize=80, cmap="YlOrRd", edgecolors='none', mincnt=2, alpha=0.6)

nyc_proj.boundary.plot(ax=ax, linewidth=0.1, edgecolor='white', alpha=0.6)

# Add basemap
cx.add_basemap(ax, source=cx.providers.CartoDB.DarkMatter, crs=nyc.crs.to_string(),

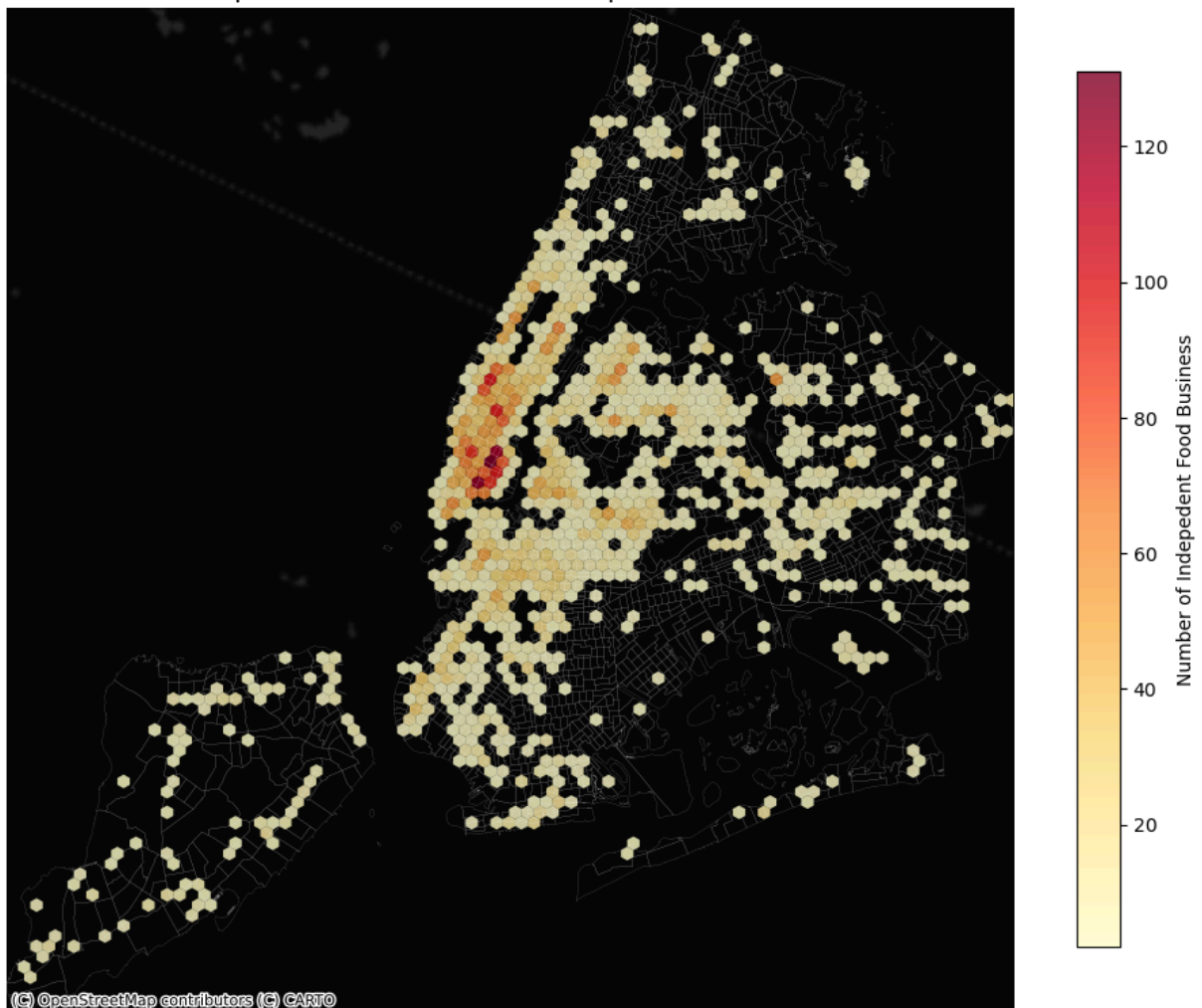
cb = fig.colorbar(hb, ax=ax, shrink=0.7, label="Number of Independent Food Business"

minx, miny, maxx, maxy = nyc_proj.total_bounds
ax.set_xlim(minx, maxx)
ax.set_ylim(miny, maxy)
ax.set_title("Independent Food Business Hotspot Hexbin in NYC", fontsize=14, color=
plt.axis("off")

```

Out[18]: (-8266094.619099499, -8204247.480376487, 4938303.227924142, 4999890.739566793)

Independent Food Business Hotspot Hexbin in NYC



Combine Census Tract data and POIS

For Food Store POIS

```
In [19]: ny_df["GEOID"] = ny_df["GEOID"].astype(str)
nyct_ny["GEOID"] = nyct_ny["GEOID"].astype(str)

nyc_geo = nyct_ny.merge(ny_df, on="GEOID", how="left")
```

```
In [20]: tract_summary = pois_nyc.groupby("GEOID").agg(
    total_pois=("name", "count"),
    chain_count=("biz_type", lambda x: (x == "chain").sum()),
    indie_count=("biz_type", lambda x: (x == "independent").sum())
).reset_index()

# Add % chain and % indie
tract_summary["pct_chain"] = tract_summary["chain_count"] / tract_summary["total_pois"]
tract_summary["pct_indie"] = tract_summary["indie_count"] / tract_summary["total_pois"]

tract_summary["pct_chain"] = tract_summary["pct_chain"].round(2)
tract_summary["pct_indie"] = tract_summary["pct_indie"].round(2)
nyc_pois_new = tract_summary.merge(nyc_geo, on="GEOID")
```

```
In [21]: nyc_pois_new.sample(10)
```

```
Out[21]:
```

	GEOID	total_pois	chain_count	indie_count	pct_chain	pct_indie	CTLabel	Boro
717	36061004700	44	9	35	20.45	79.55	47	
720	36061005000	46	18	28	39.13	60.87	50	
412	36047034700	4	2	2	50.00	50.00	347	
467	36047042300	17	2	15	11.76	88.24	423	
81	36005025300	1	0	3	0.00	300.00	253	
931	36061025300	14	9	5	64.29	35.71	253	
791	36061011402	23	11	12	47.83	52.17	114.02	
591	36047061004	6	1	6	16.67	100.00	610.04	
512	36047050001	8	0	8	0.00	100.00	500.01	
139	36005041100	1	0	1	0.00	100.00	411	

10 rows × 39 columns



For Land Use POIS

```
In [22]: nyOSM_landuse = r'C:/Users/ALIENWARE/Desktop/Machine Learning Final/new-york-latest-landuse'
landuse = gpd.read_file(nyOSM_landuse)
```

```
landuse["fclass"].unique()
```

```
Out[22]: array(['forest', 'grass', 'scrub', 'nature_reserve', 'heath', 'park',  
              'cemetery', 'retail', 'industrial', 'commercial', 'residential',  
              'recreation_ground', 'quarry', 'farmland', 'meadow', 'military',  
              'allotments', 'orchard', 'farmyard', 'vineyard'], dtype=object)
```

```
In [23]: landuse.columns
```

```
Out[23]: Index(['osm_id', 'code', 'fclass', 'name', 'geometry'], dtype='object')
```

```
In [24]: def simplify_landuse(fclass):  
          if fclass in ["residential"]:  
              return "Residential"  
          elif fclass in ["retail", "commercial"]:  
              return "Commercial"  
          elif fclass in ["industrial", "quarry"]:  
              return "Industrial"  
          elif fclass in ["park", "recreation_ground", "cemetery", "allotments", "nature_  
              return "Open_Space"  
          elif fclass in ["farmland", "meadow", "orchard", "farmyard", "vineyard"]:  
              return "Agricultural"  
          elif fclass in ["grass", "scrub", "forest"]:  
              return "Natural"  
          elif fclass in ["military"]:  
              return "Institutional"  
          else:  
              return "Other"  
  
          landuse["landuse_simple"] = landuse["fclass"].apply(simplify_landuse)  
          landuse
```

Out[24]:

	osm_id	code	fclass	name	geometry	landuse_simple
0	3751891	7201	forest	Newark Island	POLYGON ((-76.96075 43.2606, -76.96058 43.2618...	Natural
1	3751892	7201	forest	Eagle Island	POLYGON ((-76.95436 43.25909, -76.95417 43.259...	Natural
2	3751970	7218	grass	None	POLYGON ((-76.73312 43.00334, -76.73272 43.003...	Natural
3	3753148	7201	forest	Frenchman Island	POLYGON ((-76.0522 43.21948, -76.05195 43.2201...	Natural
4	3753159	7201	forest	Dunham Island	POLYGON ((-76.0412 43.21564, -76.04111 43.2156...	Natural
...	...	...	...	...	...	...
163192	1382088936	7212	retail	None	POLYGON ((-74.17997 42.93774, -74.17962 42.938...	Commercial
163193	1382091420	7204	industrial	None	POLYGON ((-74.17879 42.92718, -74.17826 42.927...	Industrial
163194	1382091421	7204	industrial	None	POLYGON ((-74.17729 42.92655, -74.17693 42.926...	Industrial
163195	19054527	7206	cemetery	Holland Cemetery	MULTIPOLYGON (((-78.55301 42.65325, -78.55189 ...	Open_Space
163196	1382105039	7203	residential	Holly Manor Apartments	POLYGON ((-73.94348 42.76854, -73.94248 42.769...	Residential

163197 rows × 6 columns

In [25]: nyc_pois_new

Out[25]:

	GEOID	total_pois	chain_count	indie_count	pct_chain	pct_indie	CTLabel	Bo
0	36005001901	7	2	5	28.57	71.43	19.01	
1	36005001902	2	0	2	0.00	100.00	19.02	
2	36005001903	3	0	3	0.00	100.00	19.03	
3	36005002500	4	1	3	25.00	75.00	25	
4	36005002800	2	2	0	100.00	0.00	28	
...	...	...	...	...	...	...	...	...
1535	36085029104	15	8	7	53.33	46.67	291.04	
1536	36085029105	9	7	2	77.78	22.22	291.05	
1537	36085030301	9	3	6	33.33	66.67	303.01	
1538	36085030302	13	3	10	23.08	76.92	303.02	
1539	36085032300	5	4	1	80.00	20.00	323	

1540 rows × 39 columns



In [26]:

```

# Match CRS
landuse = landuse.to_crs(nyct_ny.crs)

# Spatial join: clip land use polygons to each tract
landuse_ny = gpd.overlay(landuse, nyct_ny, how="intersection")

# Calculate area of each clipped land use polygon
landuse_ny = gpd.sjoin(landuse_ny, nyct_ny, how='inner', predicate='intersects')
landuse_ny["landuse_area"] = landuse_ny.geometry.area

# 5. Pivot to get area sum per land use category per tract
landuse_summary = landuse_ny.pivot_table(
    index='index_right', # or a unique census tract ID
    columns='landuse_simple', # land use category
    values='landuse_area',
    aggfunc='sum',
    fill_value=0
)

# Optional: Normalize by total area per tract
landuse_summary = landuse_summary.div(landuse_summary.sum(axis=1), axis=0)
landuse_summary

```

Out[26]:

landuse_simple	Agricultural	Commercial	Industrial	Institutional	Natural	Open_Space	...
index_right							
1	0.000000	0.000000	0.000000	0.0	0.000000	0.360727	
2	0.000000	0.000000	0.000000	0.0	0.000000	1.000000	
3	0.000000	0.000000	0.000000	0.0	0.000000	1.000000	
4	0.000000	0.000000	0.000000	0.0	0.032996	0.282129	
5	0.000000	0.000000	0.000000	0.0	1.000000	0.000000	
...	...	...	...	...	...	...	...
2316	0.000611	0.000000	0.008670	0.0	0.281877	0.708842	
2318	0.011282	0.003902	0.396754	0.0	0.045559	0.542503	
2319	0.000000	0.000000	0.961043	0.0	0.008275	0.030683	
2320	0.000000	0.000000	0.000000	0.0	0.148422	0.851578	
2324	0.000000	0.000000	0.000000	0.0	0.000000	1.000000	

1643 rows × 7 columns



In [27]:

```
nyct_ny = nyct_ny.join(landuse_summary, how="left", rsuffix="_landuse").fillna(0)
nyct_ny
```

Out[27]:

	CTLabel	BoroCode	BoroName	CT2020	BoroCT2020	CDEligibil	NTAName	NTA20
0	1	1	Manhattan	000100	1000100	I	The Battery-Governors Island-Ellis Island-Libe...	MN01
1	14.01	1	Manhattan	001401	1001401	I	Lower East Side	MN03
2	14.02	1	Manhattan	001402	1001402	E	Lower East Side	MN03
3	18	1	Manhattan	001800	1001800	I	Lower East Side	MN03
4	22.01	1	Manhattan	002201	1002201	E	Lower East Side	MN03
...	...	...	...	...	...	...	...	...
2320	86	3	Brooklyn	008600	3008600	I	Sunset Park (Central)	BK07
2321	92.01	3	Brooklyn	009201	3009201	E	Sunset Park (Central)	BK07
2322	423	3	Brooklyn	042300	3042300	E	Bushwick (West)	BK04
2323	391	3	Brooklyn	039100	3039100	E	Bushwick (West)	BK04

	CTLabel	BoroCode	BoroName	CT2020	BoroCT2020	CDEligibil	NTAName	NTA20
2324	393	3	Brooklyn	039300	3039300	E	Bushwick (West)	BK04

2325 rows × 22 columns

Appending All POIS

In [28]: `nyct_ny.head(1)`

Out[28]:

	CTLabel	BoroCode	BoroName	CT2020	BoroCT2020	CDEligibil	NTAName	NTA2020
0	1	1	Manhattan	000100	1000100	I	The Battery-Governors Island-Ellis Island-Libe...	MN0191

1 rows × 22 columns



In [29]:

```

nyc_pois_new["GEOID"] = nyc_pois_new["GEOID"].astype(str)
nyct_ny["GEOID"] = nyct_ny["GEOID"].astype(str)

# Select only Land use columns from nyct_ny (including GEOID as key)
landuse_cols = ['GEOID', 'Agricultural', 'Commercial', 'Industrial', 'Institutional',
                 'Natural', 'Open_Space', 'Residential']

landuse_only = nyct_ny[landuse_cols].copy()

# Merge Land use info
nyc_pois_new = nyc_pois_new.merge(landuse_only, on="GEOID", how="left")
nyc_pois_new.fillna(0, inplace=True)

```

In [30]: `nyc_pois_new`

Out[30]:

	GEOID	total_pois	chain_count	indie_count	pct_chain	pct_indie	CTLabel	Bo
0	36005001901	7	2	5	28.57	71.43	19.01	
1	36005001902	2	0	2	0.00	100.00	19.02	
2	36005001903	3	0	3	0.00	100.00	19.03	
3	36005002500	4	1	3	25.00	75.00	25	
4	36005002800	2	2	0	100.00	0.00	28	
...	...	...	...	...	...	...	...	...
1535	36085029104	15	8	7	53.33	46.67	291.04	
1536	36085029105	9	7	2	77.78	22.22	291.05	
1537	36085030301	9	3	6	33.33	66.67	303.01	
1538	36085030302	13	3	10	23.08	76.92	303.02	
1539	36085032300	5	4	1	80.00	20.00	323	

1540 rows × 46 columns



Simple classification test (not ML)

```
In [31]: q1 = nyc_pois_new["pct_chain"].quantile(0.25)
q3 = nyc_pois_new["pct_chain"].quantile(0.75)

def classify_cluster(pct_chain):
    if pct_chain >= q3:
        return "chain_prone_zone"
    elif pct_chain <= q1:
        return "indie_prone_zone"
    else:
        return "mixed_zone"

nyc_clustered = nyc_pois_new.copy()
nyc_clustered["cluster_label"] = nyc_clustered["pct_chain"].apply(classify_cluster)
```

```
In [32]: nyc_clustered
```

Out[32]:

	GEOID	total_pois	chain_count	indie_count	pct_chain	pct_indie	CTLabel	Bo
0	36005001901	7	2	5	28.57	71.43	19.01	
1	36005001902	2	0	2	0.00	100.00	19.02	
2	36005001903	3	0	3	0.00	100.00	19.03	
3	36005002500	4	1	3	25.00	75.00	25	
4	36005002800	2	2	0	100.00	0.00	28	
...	...	...	...	...	...	...	...	...
1535	36085029104	15	8	7	53.33	46.67	291.04	
1536	36085029105	9	7	2	77.78	22.22	291.05	
1537	36085030301	9	3	6	33.33	66.67	303.01	
1538	36085030302	13	3	10	23.08	76.92	303.02	
1539	36085032300	5	4	1	80.00	20.00	323	

1540 rows × 47 columns



```
In [33]: label_colors = {
    "chain_prone_zone": "#FA8072",
    "indie_prone_zone": "#7EC8E3",
    "mixed_zone" : "#FFC300"
}
nyc_clustered["color"] = nyc_clustered["cluster_label"].map(label_colors)
```

```
In [34]: nyc_clustered_ny = nyc_clustered[nyc_clustered["BoroName"].isin(['Manhattan', 'Bro
```

```
In [35]: import matplotlib.patches as mpatches

nyc_clustered_ny = gpd.GeoDataFrame(nyc_clustered_ny, geometry='geometry', crs='EPS
fig, ax = plt.subplots(figsize = (12,10))

nyct_ny.plot(ax=ax, edgecolor="grey", facecolor="none", linewidth=0.5)

for label, color in label_colors.items():
    nyc_clustered_ny[nyc_clustered_ny["cluster_label"] == label].plot(
        ax=ax,
        color=color,
        label=label.replace("_", " ").title(),
        edgecolor="none",
        linewidth=0
```

```

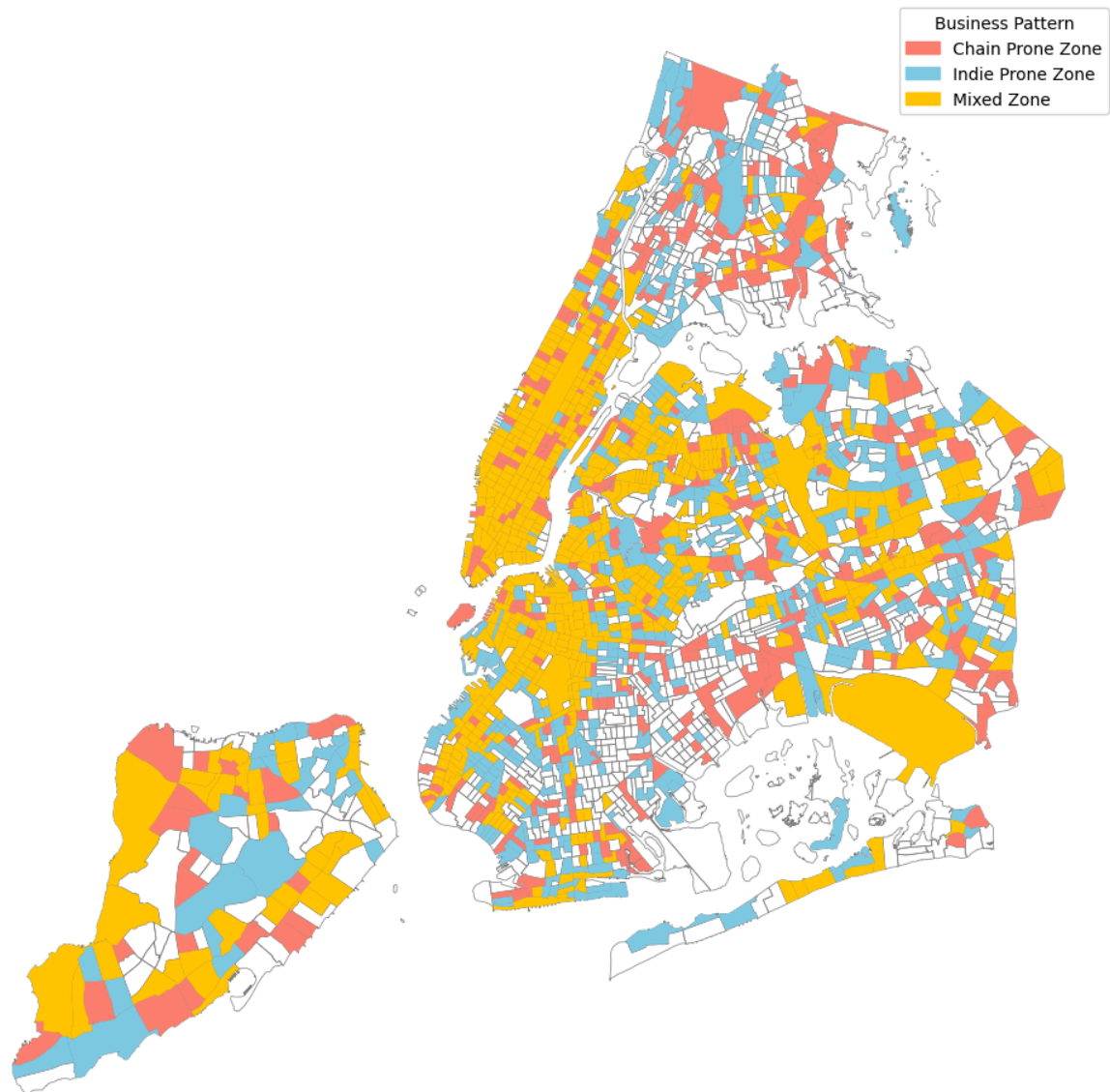
)

legend_patches = [
    mpatches.Patch(color=color, label=label.replace("_", " ").title())
    for label, color in label_colors.items()
]
plt.legend(handles=legend_patches, title="Business Pattern")

plt.title("Store Type Classification by Census Tract", fontsize=14)
plt.axis("off")
plt.tight_layout()
plt.show()

```

Store Type Classification by Census Tract



K-Means Cluster Analysis of NYC Census Tract Data

Data organizing

```
In [36]: nyc_pois_new["cluster_label"] = nyc_pois_new["pct_chain"].apply(classify_cluster)
nyc_pois_new.sample(5)
```

```
Out[36]:
```

	GEOID	total_pois	chain_count	indie_count	pct_chain	pct_indie	CTLabel	Bo
382	36047029500	13	1	12	7.69	92.31	295	
1272	36081059100	5	0	5	0.00	100.00	591	
208	36047006500	20	3	17	15.00	85.00	65	
1411	36081120500	6	0	6	0.00	100.00	1205	
1185	36081041500	6	2	4	33.33	66.67	415	

5 rows × 47 columns



```
In [37]: nyc_pois_final = nyc_pois_new.dropna()
```

```
In [38]: nyc_pois_final.columns
```

```
Out[38]: Index(['GEOID', 'total_pois', 'chain_count', 'indie_count', 'pct_chain',
               'pct_indie', 'CTLabel', 'BoroCode', 'BoroName', 'CT2020', 'BoroCT2020',
               'CDEligibil', 'NTAName', 'NTA2020', 'CDTA2020', 'CDTANAME', 'PUMA',
               'Shape_Leng', 'Shape_Area', 'geometry', 'NAME', 'B01003_001E',
               'B19013_001E', 'B17001_001E', 'B17001_002E', 'B23025_003E',
               'B23025_005E', 'B25003_001E', 'B25044_003E', 'B08301_001E',
               'B08301_010E', 'state', 'county', 'tract', 'total_population',
               'median_income', 'poverty_rate', 'unemployment_rate', 'no_vehicle_rate',
               'Agricultural', 'Commercial', 'Industrial', 'Institutional', 'Natural',
               'Open_Space', 'Residential', 'cluster_label'],
              dtype='object')
```

```
In [39]: nyc_pois_final["GEOID"] = nyc_pois_final["GEOID"].astype(str)
nyct_ny["GEOID"] = nyct_ny["GEOID"].astype(str)
```

```
# Project geometry to meters and calculate area in km²
nyct_ny_area = nyct_ny.set_index("GEOID").copy()
nyct_ny_area = nyct_ny_area.to_crs(epsg=6539)
```

```
In [40]: # Make sure tract area is in square kilometers
nyc_pois_final["tract_area_km2"] = nyct_ny.set_index("GEOID").loc[nyc_pois_final["GEOID"]].area

# Normalize POI counts by area
nyc_pois_final["pois_per_km2"] = nyc_pois_final["total_pois"] / nyc_pois_final["tract_area_km2"]
nyc_pois_final["chain_per_km2"] = nyc_pois_final["chain_count"] / nyc_pois_final["tract_area_km2"]
nyc_pois_final["indie_per_km2"] = nyc_pois_final["indie_count"] / nyc_pois_final["tract_area_km2"]
```

```
In [41]: nyc_pois_final
```

Out[41]:

	GEOID	total_pois	chain_count	indie_count	pct_chain	pct_indie	CTLabel	Bo
0	36005001901	7	2	5	28.57	71.43	19.01	
1	36005001902	2	0	2	0.00	100.00	19.02	
2	36005001903	3	0	3	0.00	100.00	19.03	
3	36005002500	4	1	3	25.00	75.00	25	
4	36005002800	2	2	0	100.00	0.00	28	
...	...	...	...	...	...	...	...	...
1535	36085029104	15	8	7	53.33	46.67	291.04	
1536	36085029105	9	7	2	77.78	22.22	291.05	
1537	36085030301	9	3	6	33.33	66.67	303.01	
1538	36085030302	13	3	10	23.08	76.92	303.02	
1539	36085032300	5	4	1	80.00	20.00	323	

1540 rows × 51 columns

In [42]: `from collections import Counter`

```

def classify_cluster_by_weighted(indie_per_km2, chain_per_km2, threshold=0.3):
    diff = indie_per_km2 - chain_per_km2
    if diff > threshold:
        return "indie_prone_zone"
    elif diff < -threshold:
        return "chain_prone_zone"
    else:
        return "mixed_zone"

nyc_pois_final["cluster_label_weighted"] = nyc_pois_final.apply(
    lambda row: classify_cluster_by_weighted(
        row["indie_per_km2"], row["chain_per_km2"], threshold=0.2
    ), axis=1)

# Count how many in each label
zone_counts = Counter(nyc_pois_final["cluster_label_weighted"])
print(zone_counts)

```

Counter({'indie_prone_zone': 1118, 'mixed_zone': 214, 'chain_prone_zone': 208})

In [43]: `nyc_pois_final.sample(5)`

Out[43]:

	GEOID	total_pois	chain_count	indie_count	pct_chain	pct_indie	CTLabel	Bo
1385	36081100803	3	1	2	33.33	66.67	1008.03	
488	36047044902	4	0	4	0.00	100.00	449.02	
1094	36081022002	1	0	1	0.00	100.00	220.02	
963	36081001200	11	2	9	18.18	81.82	12	
818	36061013603	4	1	3	25.00	75.00	136.03	

5 rows × 52 columns



In [44]:

```
import matplotlib.patches as mpatches

label_colors = {
    "chain_prone_zone": "#FA8072",
    "indie_prone_zone": "#7EC8E3",
    "mixed_zone": "#FFC300"
}

nyc_pois_final["color"] = nyc_pois_final["cluster_label_weighted"].map(label_colors)

nyc_pois_final = gpd.GeoDataFrame(nyc_pois_final, geometry='geometry', crs='EPSG:22

fig, ax = plt.subplots(figsize = (12,10))

nyct_ny.plot(ax=ax, edgecolor="grey", facecolor="none", linewidth=0.5)

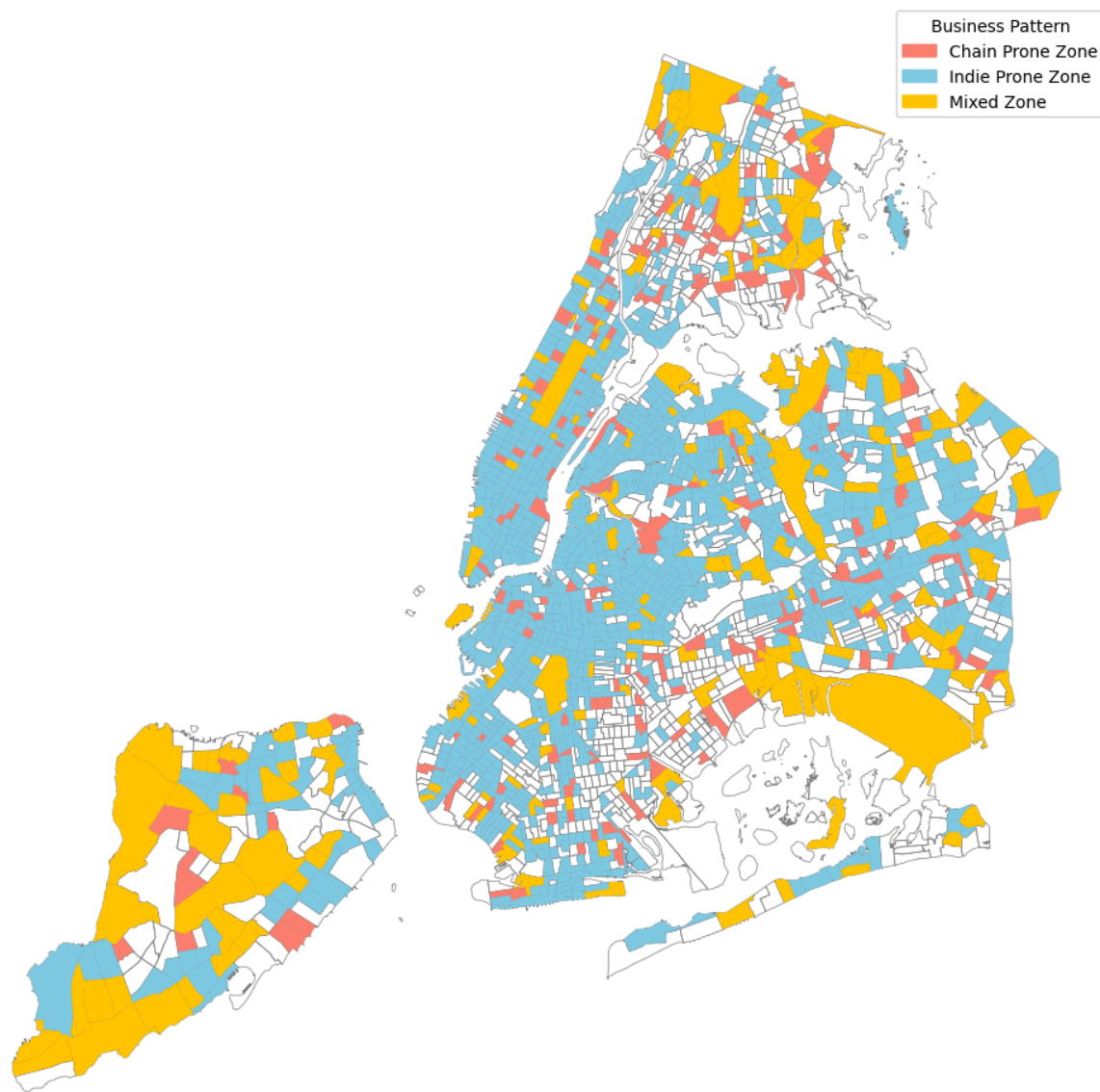
for label, color in label_colors.items():
    nyc_pois_final[nyc_pois_final["cluster_label_weighted"] == label].plot(
        ax=ax,
        color=color,
        label=label.replace("_", " ").title(),
        edgecolor="none",
        linewidth=0
    )

legend_patches = [
    mpatches.Patch(color=color, label=label.replace("_", " ").title())
    for label, color in label_colors.items()
]

plt.legend(handles=legend_patches, title="Business Pattern")

plt.title("Store Type Classification (weighted) by Census Tract", fontsize=14)
plt.axis("off")
plt.tight_layout()
plt.show()
```

Store Type Classification (weighted) by Census Tract



Select Feature for inclusion in cluster analysis

```
In [45]: cluster_df = nyc_pois_final[['total_population', 'indie_per_km2', 'poverty_rate', 'no_
display(cluster_df)
```

	total_population	indie_per_km2	poverty_rate	no_vehicle_rate	Open_Space	Residentia
0	2181.0	2.239568	28.604224	0.000000	0.127073	0.000000
1	1883.0	0.395318	32.304300	2.991453	0.266199	0.000000
2	0.0	0.308061	0.000000	0.000000	0.000000	0.000000
3	5642.0	2.186348	44.488049	4.629630	0.041700	0.958300
4	5230.0	0.000000	2.503370	2.084972	0.759542	0.157120
...	...	...	...	...	...	...
1535	6875.0	0.421570	6.894545	2.634829	0.447886	0.108390
1536	4666.0	0.115396	7.308187	3.267571	0.471921	0.008480
1537	5915.0	0.772527	3.147208	1.627219	0.022740	0.537660
1538	6374.0	1.021640	14.060778	5.204819	0.593387	0.250440
1539	1133.0	0.024024	4.942630	6.117647	0.262682	0.011010

1540 rows × 6 columns



```
In [46]: cluster_df.replace([np.inf, -np.inf], np.nan, inplace=True)
cluster_df = cluster_df.dropna()

from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()

X = scaler.fit_transform(cluster_df)
display(pd.DataFrame(X, columns=cluster_df.columns))
```

C:\Users\ALIENWARE\AppData\Local\Temp\ipykernel_29148\1794331471.py:1: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
cluster_df.replace([np.inf, -np.inf], np.nan, inplace=True)

	total_population	indie_per_km2	poverty_rate	no_vehicle_rate	Open_Space	Residentia
0	0.136783	0.034874	0.286042	0.000000	0.127073	0.000000
1	0.118093	0.006156	0.323043	0.057585	0.266199	0.000000
2	0.000000	0.004797	0.000000	0.000000	0.000000	0.000000
3	0.353841	0.034045	0.444880	0.089120	0.041700	0.958300
4	0.328003	0.000000	0.025034	0.040136	0.759542	0.157120
...	...	...	...	...	...	...
1535	0.431170	0.006565	0.068945	0.050720	0.447886	0.108390
1536	0.292631	0.001797	0.073082	0.062901	0.471921	0.008480
1537	0.370963	0.012030	0.031472	0.031324	0.022740	0.537660
1538	0.399749	0.015909	0.140608	0.100193	0.593387	0.250440
1539	0.071057	0.000374	0.049426	0.117765	0.262682	0.011010

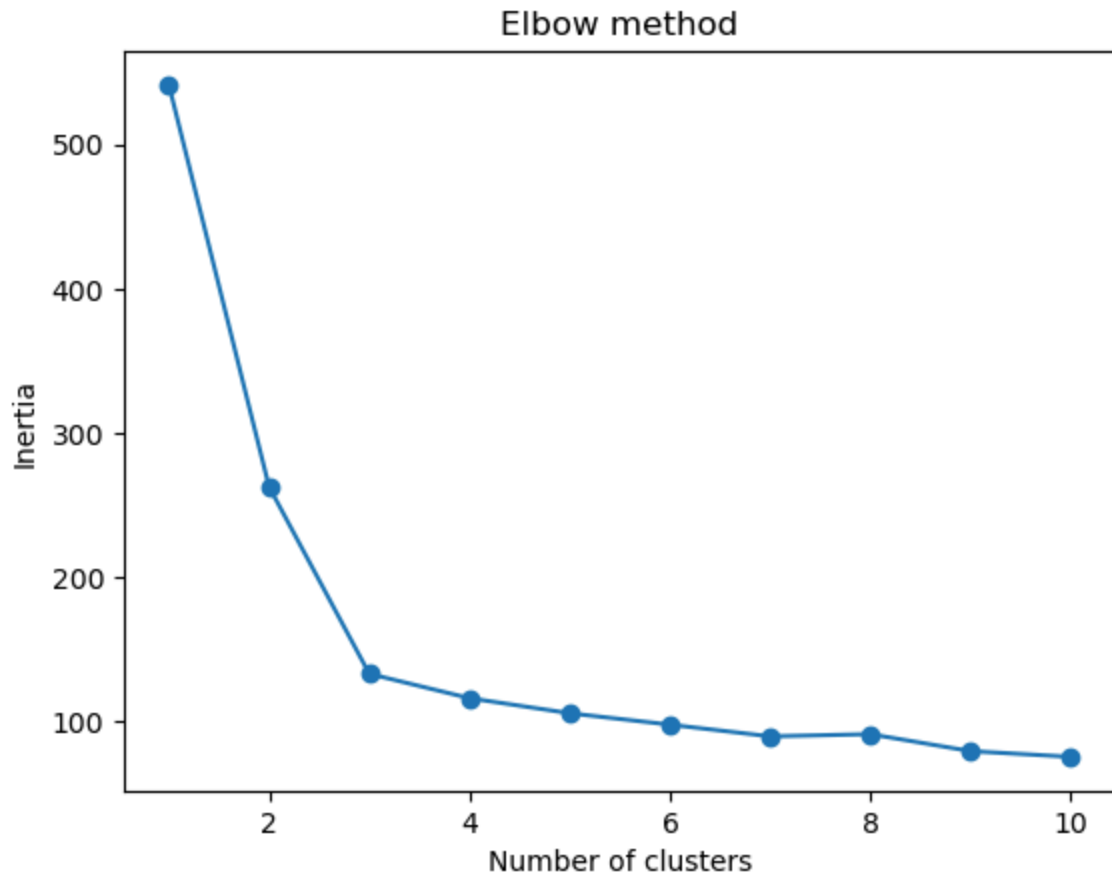
1540 rows × 6 columns



Diagnostic texting for a defensible number of cluster

```
In [47]: from sklearn.cluster import KMeans
inertias = []
for i in range(1,11):
    kmeans_elbow = KMeans(n_clusters=i, n_init="auto")
    kmeans_elbow.fit(X)
    inertias.append(kmeans_elbow.inertia_)
plt.plot(range(1,11), inertias, marker='o')
plt.title('Elbow method')
plt.xlabel('Number of clusters')
plt.ylabel('Inertia')
plt.show()
```

```
D:\Anaconda\Lib\site-packages\sklearn\cluster\_kmeans.py:1382: UserWarning: KMeans i  
s known to have a memory leak on Windows with MKL, when there are less chunks than a  
vailable threads. You can avoid it by setting the environment variable OMP_NUM_THREA  
DS=7.  
warnings.warn(  
D:\Anaconda\Lib\site-packages\sklearn\cluster\_kmeans.py:1382: UserWarning: KMeans i  
s known to have a memory leak on Windows with MKL, when there are less chunks than a  
vailable threads. You can avoid it by setting the environment variable OMP_NUM_THREA  
DS=7.  
warnings.warn(  
D:\Anaconda\Lib\site-packages\sklearn\cluster\_kmeans.py:1382: UserWarning: KMeans i  
s known to have a memory leak on Windows with MKL, when there are less chunks than a  
vailable threads. You can avoid it by setting the environment variable OMP_NUM_THREA  
DS=7.  
warnings.warn(  
D:\Anaconda\Lib\site-packages\sklearn\cluster\_kmeans.py:1382: UserWarning: KMeans i  
s known to have a memory leak on Windows with MKL, when there are less chunks than a  
vailable threads. You can avoid it by setting the environment variable OMP_NUM_THREA  
DS=7.  
warnings.warn(  
D:\Anaconda\Lib\site-packages\sklearn\cluster\_kmeans.py:1382: UserWarning: KMeans i  
s known to have a memory leak on Windows with MKL, when there are less chunks than a  
vailable threads. You can avoid it by setting the environment variable OMP_NUM_THREA  
DS=7.  
warnings.warn(  
D:\Anaconda\Lib\site-packages\sklearn\cluster\_kmeans.py:1382: UserWarning: KMeans i  
s known to have a memory leak on Windows with MKL, when there are less chunks than a  
vailable threads. You can avoid it by setting the environment variable OMP_NUM_THREA  
DS=7.  
warnings.warn(  
D:\Anaconda\Lib\site-packages\sklearn\cluster\_kmeans.py:1382: UserWarning: KMeans i  
s known to have a memory leak on Windows with MKL, when there are less chunks than a  
vailable threads. You can avoid it by setting the environment variable OMP_NUM_THREA  
DS=7.  
warnings.warn(  
D:\Anaconda\Lib\site-packages\sklearn\cluster\_kmeans.py:1382: UserWarning: KMeans i  
s known to have a memory leak on Windows with MKL, when there are less chunks than a  
vailable threads. You can avoid it by setting the environment variable OMP_NUM_THREA  
DS=7.  
warnings.warn(  
D:\Anaconda\Lib\site-packages\sklearn\cluster\_kmeans.py:1382: UserWarning: KMeans i  
s known to have a memory leak on Windows with MKL, when there are less chunks than a  
vailable threads. You can avoid it by setting the environment variable OMP_NUM_THREA  
DS=7.  
warnings.warn(  
D:\Anaconda\Lib\site-packages\sklearn\cluster\_kmeans.py:1382: UserWarning: KMeans i  
s known to have a memory leak on Windows with MKL, when there are less chunks than a  
vailable threads. You can avoid it by setting the environment variable OMP_NUM_THREA  
DS=7.
```



Running the final K-Mean cluster analysis

```
In [48]: # Conduct the K-means analysis
n_k = 3
from sklearn.cluster import KMeans
kmeans_model = KMeans(n_clusters=n_k, random_state=0, n_init="auto").fit(X)
# Display out cluster center means
display(pd.DataFrame(np.round(kmeans_model.cluster_centers_, decimals=4), columns=c1
```

D:\Anaconda\Lib\site-packages\sklearn\cluster_kmeans.py:1382: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than a available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=7.

```
warnings.warn(
```

	total_population	indie_per_km2	poverty_rate	no_vehicle_rate	Open_Space	Residential
0	0.2969	0.0362	0.1809	0.1664	0.0833	0.8450
1	0.2660	0.0685	0.1471	0.1748	0.8955	0.0147
2	0.2238	0.0601	0.1533	0.1725	0.0352	0.0119

```
In [49]: # Predict and visualize the clusters using pairs of features
y_label = kmeans_model.fit_predict(X)
print(y_label)
```

```
# looping to plot clusters for pairs of features
columnx = 0
columny = 1

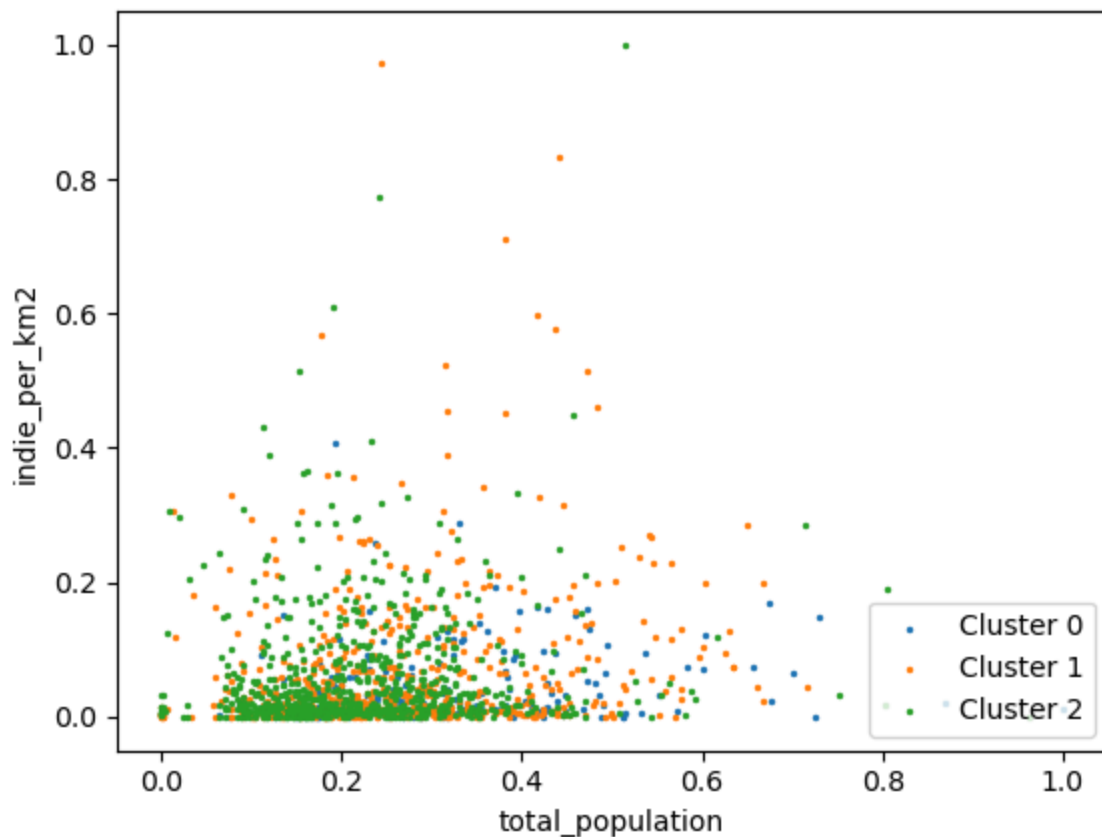
for i in range(0,n_k):
    plt.scatter(X[y_label==i, columnx], X[y_label==i, columny], s=2, label='Cluster'

plt.xlabel(cluster_df.columns[columnx])
plt.ylabel(cluster_df.columns[columny])
plt.legend(loc="lower right")
plt.show()
```

D:\Anaconda\Lib\site-packages\sklearn\cluster_kmeans.py:1382: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than a available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=7.

warnings.warn(

[2 2 2 ... 0 1 2]



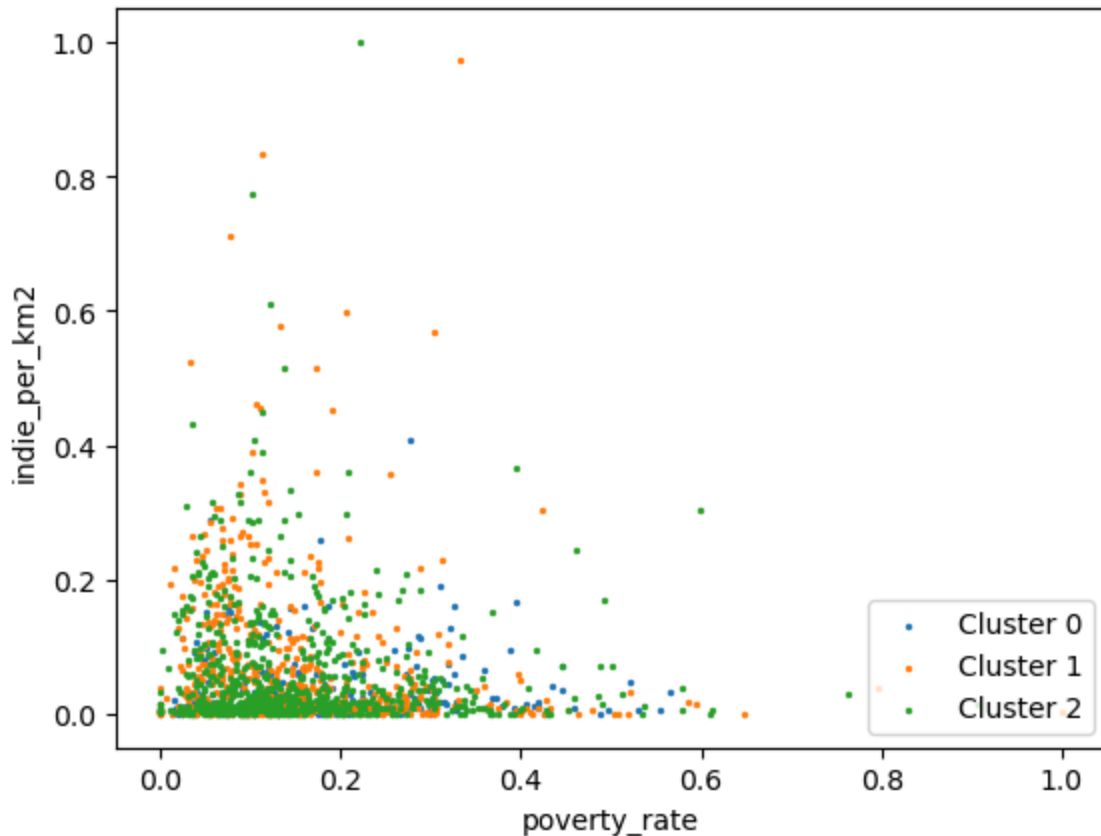
```
In [50]: # Predict and visualize the clusters using pairs of features
y_label = kmeans_model.fit_predict(X)
print(y_label)

# looping to plot clusters for pairs of features
columnx = 2
columny = 1
0
for i in range(0,n_k):
    plt.scatter(X[y_label==i, columnx], X[y_label==i, columny], s=2, label='Cluster
```

```
plt.xlabel(cluster_df.columns[columnx])
plt.ylabel(cluster_df.columns[columny])
plt.legend(loc="lower right")
plt.show()
```

D:\Anaconda\Lib\site-packages\sklearn\cluster_kmeans.py:1382: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than a available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=7.

```
warnings.warn(
[2 2 2 ... 0 1 2]
```



```
In [51]: # Predict and visualize the clusters using pairs of features
y_label = kmeans_model.fit_predict(X)
print(y_label)

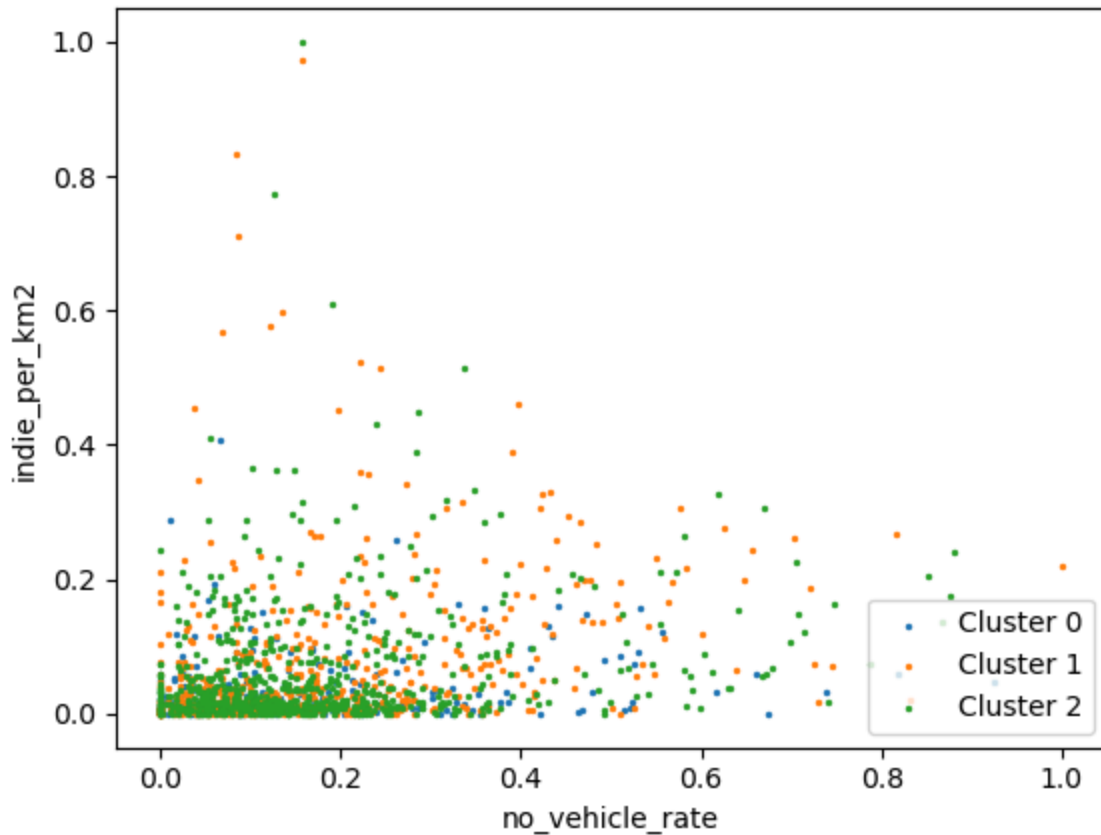
# looping to plot clusters for pairs of features
columnx = 3
columny = 1

for i in range(0,n_k):
    plt.scatter(X[y_label==i, columnx], X[y_label==i, columny], s=2, label='Cluster ' + str(i))

plt.xlabel(cluster_df.columns[columnx])
plt.ylabel(cluster_df.columns[columny])
plt.legend(loc="lower right")
plt.show()
```

D:\Anaconda\Lib\site-packages\sklearn\cluster_kmeans.py:1382: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than a available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=7.

```
warnings.warn(
[2 2 2 ... 0 1 2]
```



```
In [52]: # Predict and visualize the clusters using pairs of features
y_label = kmeans_model.fit_predict(X)
print(y_label)

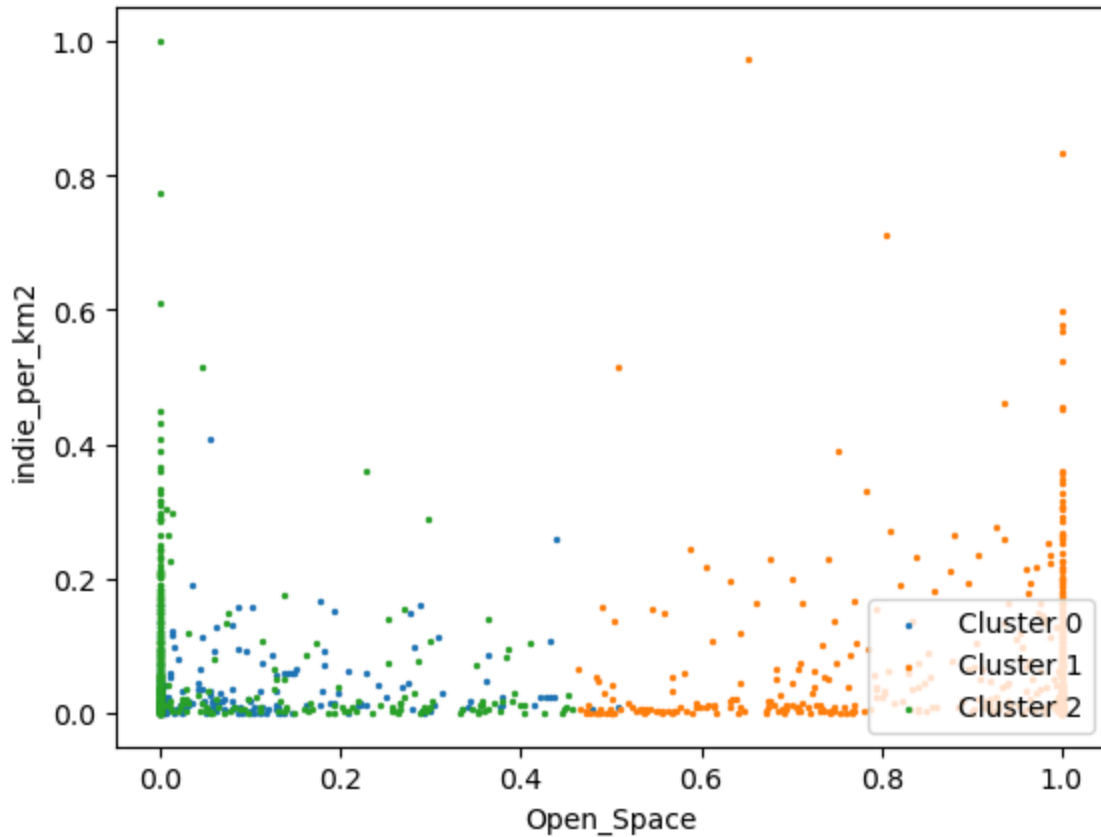
# Looping to plot clusters for pairs of features
columnx = 4
columny = 1

for i in range(0,n_k):
    plt.scatter(X[y_label==i, columnx], X[y_label==i, columny], s=2, label='Cluster ' + str(i))

plt.xlabel(cluster_df.columns[columnx])
plt.ylabel(cluster_df.columns[columny])
plt.legend(loc="lower right")
plt.show()
```

D:\Anaconda\Lib\site-packages\sklearn\cluster_kmeans.py:1382: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than a available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=7.

```
warnings.warn(
[2 2 2 ... 0 1 2]
```



```
In [53]: # Predict and visualize the clusters using pairs of features
y_label = kmeans_model.fit_predict(X)
print(y_label)

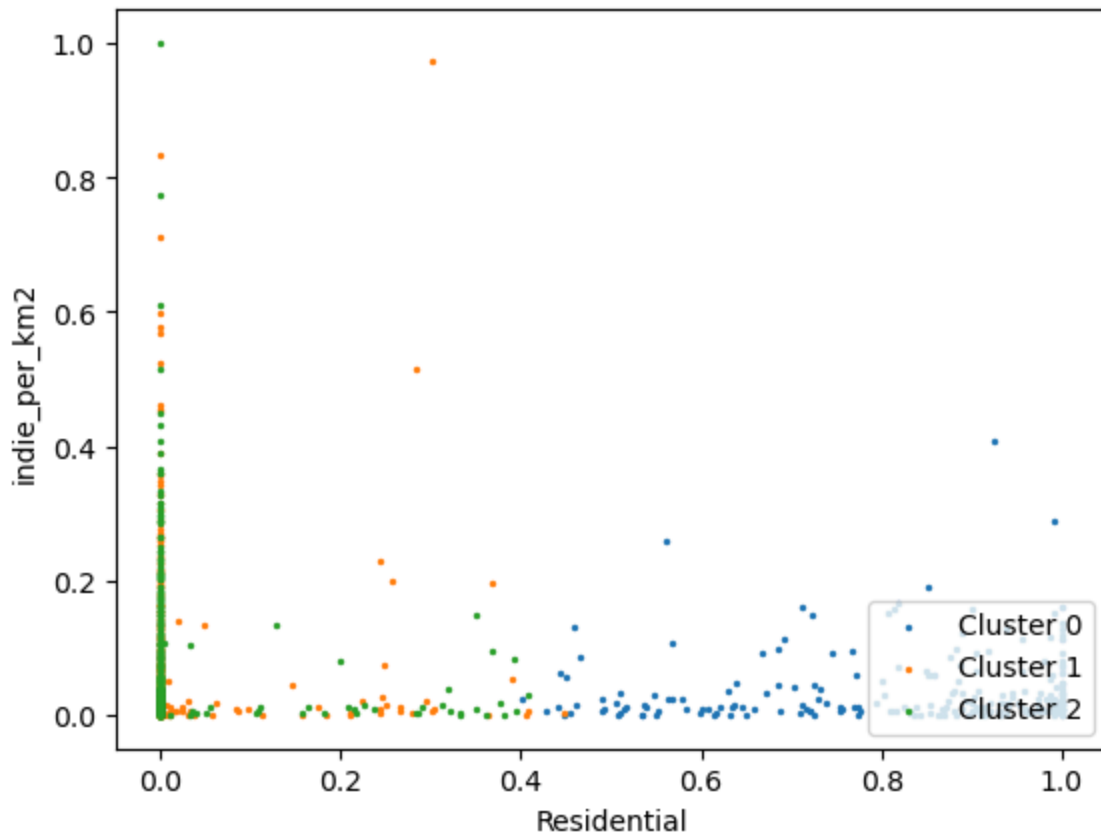
# looping to plot clusters for pairs of features
columnx = 5
columny = 1

for i in range(0,n_k):
    plt.scatter(X[y_label==i, columnx], X[y_label==i, columny], s=2, label='Cluster ' + str(i))

plt.xlabel(cluster_df.columns[columnx])
plt.ylabel(cluster_df.columns[columny])
plt.legend(loc="lower right")
plt.show()
```

D:\Anaconda\Lib\site-packages\sklearn\cluster_kmeans.py:1382: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than a available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=7.

```
warnings.warn(
[2 2 2 ... 0 1 2])
```



```
In [54]: # Visualize the tree (dendrogram) for this data
from sklearn.cluster import AgglomerativeClustering
agglom_model = AgglomerativeClustering(distance_threshold=0, n_clusters=None)
agglom_model = agglom_model.fit(X)

from scipy.cluster.hierarchy import dendrogram
def plot_dendrogram(model, **kwargs):

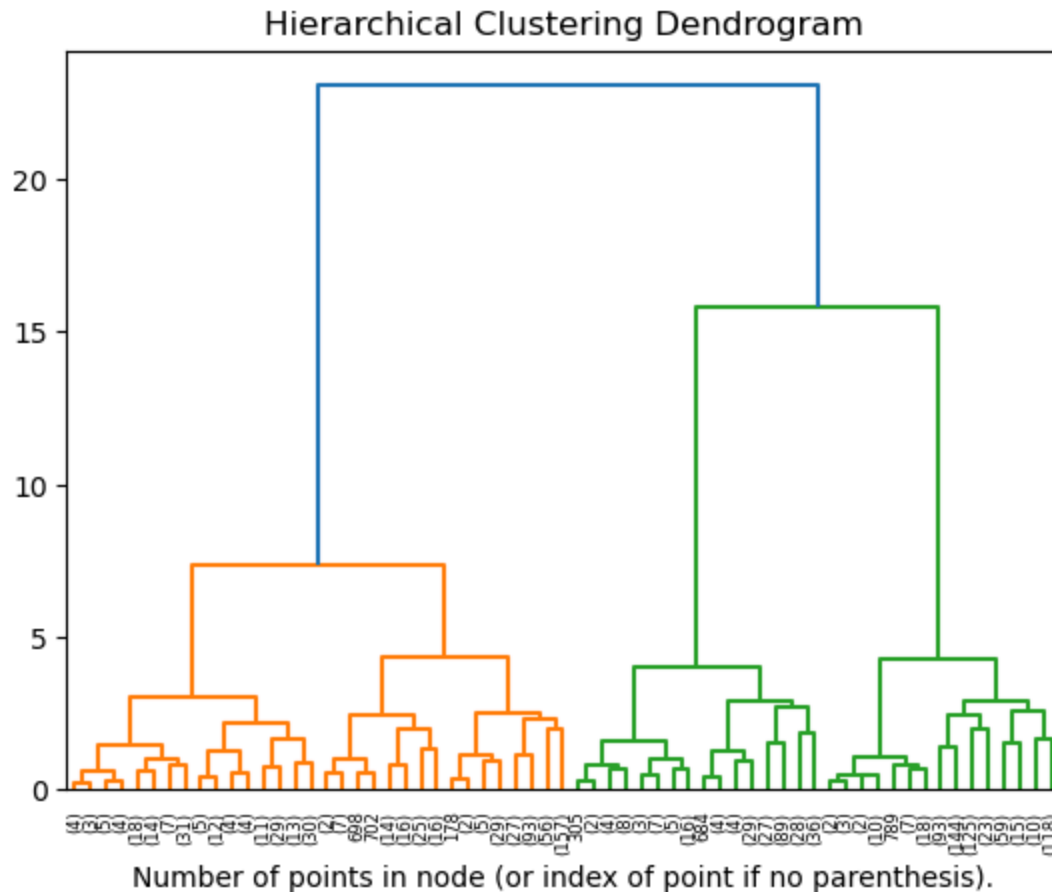
    counts = np.zeros(model.children_.shape[0])
    n_samples = len(model.labels_)
    for i, merge in enumerate(model.children_):
        current_count = 0
        for child_idx in merge:
            if child_idx < n_samples:
                current_count += 1 # Leaf node
            else:
                current_count += counts[child_idx - n_samples]
        counts[i] = current_count

    linkage_matrix = np.column_stack([model.children_, model.distances_, counts]).a

    dendrogram(linkage_matrix, **kwargs)

plt.title("Hierarchical Clustering Dendrogram")

# plot the top three levels of the dendrogram
plot_dendrogram(agglom_model, truncate_mode="level", p=5)
plt.xlabel("Number of points in node (or index of point if no parenthesis).")
plt.show()
```

Classification

```
In [55]: # Feed data from a DataFrame into train_test_split
from sklearn.model_selection import train_test_split

features = ['total_population', 'poverty_rate', 'no_vehicle_rate', 'Open_Space', 'Resid

X = nyc_pois_final[features]
y = nyc_pois_final["cluster_label_weighted"] # Make sure this column has the 3 cat

X = X.dropna()
y = y.loc[X.index]

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)
```

RandomForest Classifier

```
In [56]: # Random Forest Classifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix
import seaborn as sns
```

```

clf = RandomForestClassifier(n_estimators=100, random_state=42)
clf.fit(X_train, y_train)

# Prediction and Evaluation
y_pred = clf.predict(X_test)

print("\nClassification Report:")
print(classification_report(y_test, y_pred))

print("Confusion Matrix:")
cm = confusion_matrix(y_test, y_pred, labels=clf.classes_)
sns.heatmap(cm, annot=True, fmt="d", xticklabels=clf.classes_, yticklabels=clf.classes_)
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()

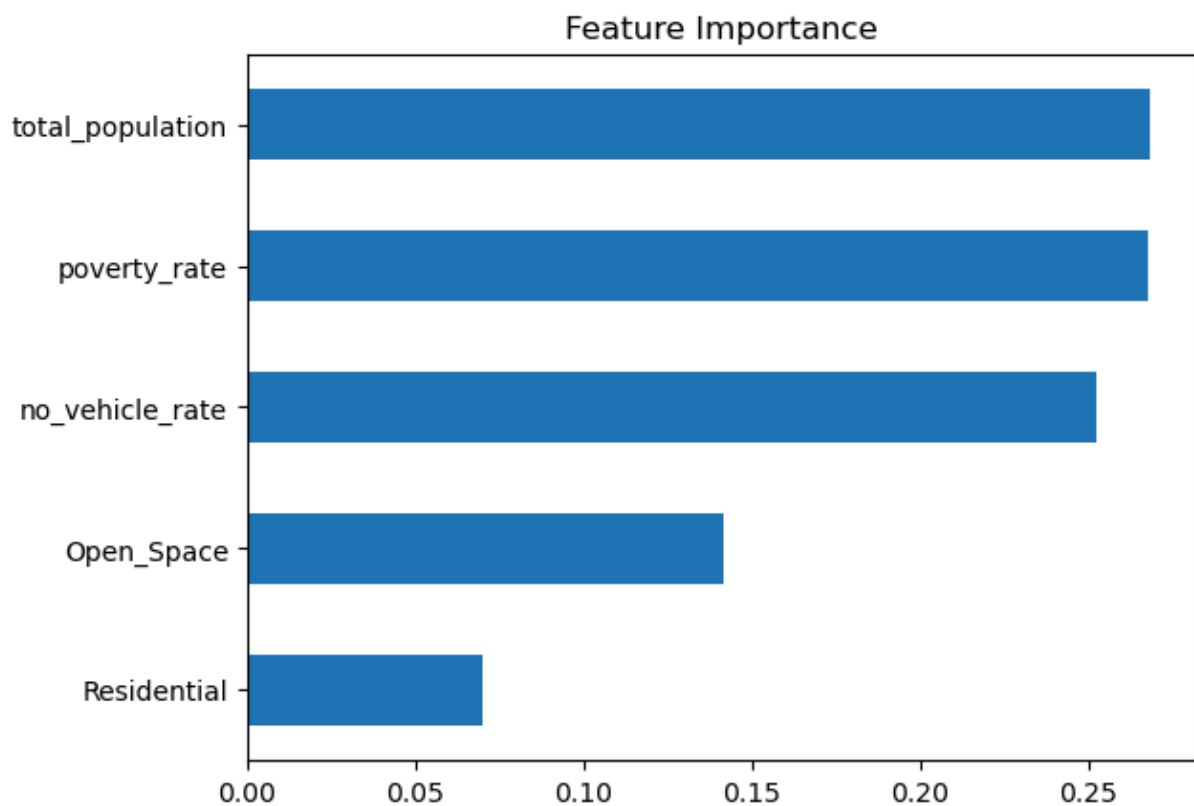
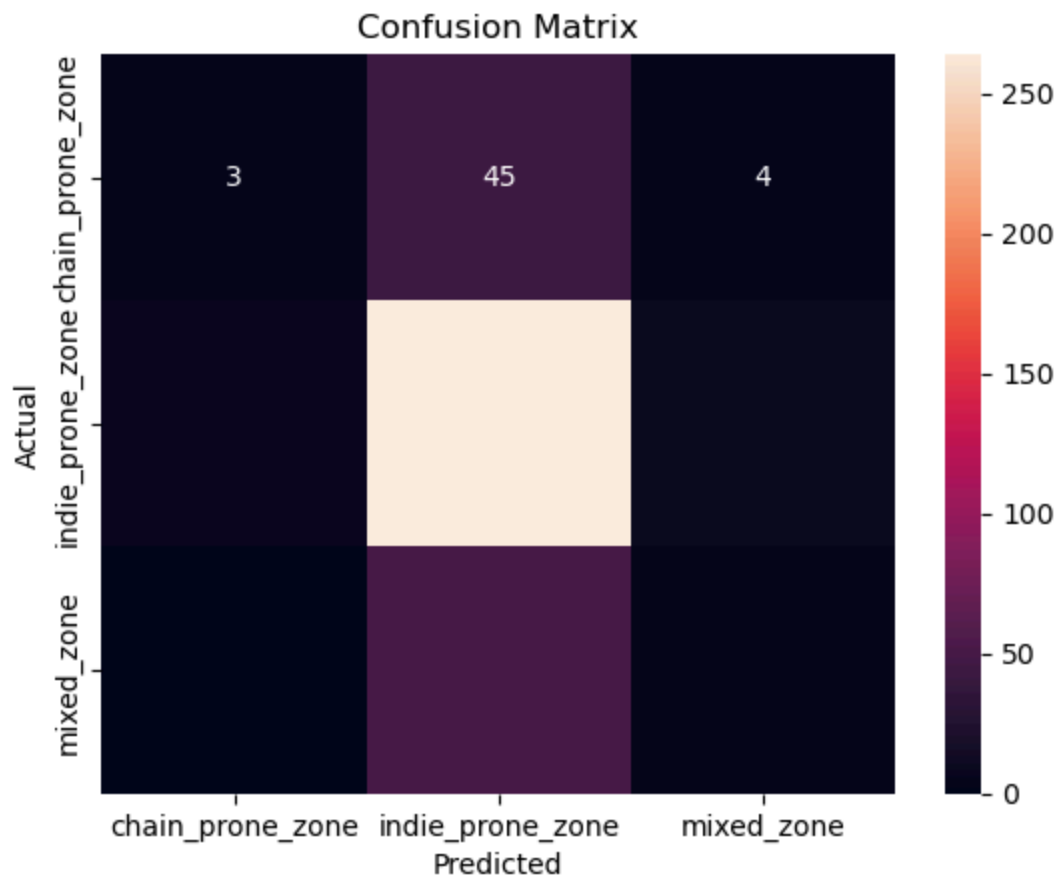
# Feature importance
importances = pd.Series(clf.feature_importances_, index=features)
importances.sort_values().plot(kind="barh", title="Feature Importance")
plt.show()

```

Classification Report:

	precision	recall	f1-score	support
chain_prone_zone	0.30	0.06	0.10	52
indie_prone_zone	0.74	0.94	0.83	280
mixed_zone	0.24	0.08	0.11	53
accuracy			0.70	385
macro avg	0.42	0.36	0.35	385
weighted avg	0.61	0.70	0.63	385

Confusion Matrix:



K-Nearest Neighbor Classifier

```
In [57]: # After importing the sklearn.neighbors object we can fit our model
from sklearn.neighbors import KNeighborsClassifier
knn = KNeighborsClassifier(n_neighbors=6)
knn.fit(X_train, y_train)

# To evaluate the model we need to make predictions for our test data
y_pred = knn.predict(X_test)
print("Test set predictions:\n", y_pred)
print("Actual values:\n", y_test)
# Based on that we can calculate the accuracy of our model
print("Test set score: ", knn.score(X_test, y_test))

# a confusion matrix compares the actual test data target values with what the mode
# the results form with matrix of true positive, false positive on top
# and false negative, true negative positive on bottom
from sklearn.metrics import confusion_matrix
print(confusion_matrix(y_test, y_pred))
```


38/40

```
'indie_prone_zone' 'indie_prone_zone' 'indie_prone_zone'
'indie_prone_zone' 'indie_prone_zone' 'indie_prone_zone'
'indie_prone_zone' 'indie_prone_zone' 'indie_prone_zone'
'indie_prone_zone' 'chain_prone_zone' 'indie_prone_zone'
'indie_prone_zone' 'indie_prone_zone' 'indie_prone_zone'
'indie_prone_zone' 'indie_prone_zone' 'chain_prone_zone'
'indie_prone_zone' 'indie_prone_zone' 'indie_prone_zone'
'chain_prone_zone' 'indie_prone_zone' 'indie_prone_zone'
'indie_prone_zone' 'indie_prone_zone' 'mixed_zone' 'indie_prone_zone'
'indie_prone_zone' 'indie_prone_zone' 'indie_prone_zone'
'indie_prone_zone' 'indie_prone_zone' 'indie_prone_zone'
'indie_prone_zone' 'indie_prone_zone' 'indie_prone_zone'
'indie_prone_zone' 'chain_prone_zone' 'indie_prone_zone']
```

Actual values:

```
213      indie_prone_zone
623      indie_prone_zone
1483      mixed_zone
717      indie_prone_zone
255      indie_prone_zone
...
```

```
81      indie_prone_zone
533      indie_prone_zone
939      indie_prone_zone
1502     indie_prone_zone
114      indie_prone_zone
```

Name: cluster_label_weighted, Length: 385, dtype: object

Test set score: 0.7012987012987013

```
[[ 1 51  0]
 [10 263 7]
 [ 3 44 6]]
```

Logistic Regression Classifier

```
In [58]: # Fit our Logistic Regression training model
# Make predictions, view scores and coefficients
from sklearn.linear_model import LogisticRegression
logreg = LogisticRegression(C=10,max_iter=10000,).fit(X_train, y_train)
print("Training set score: {:.3f}".format(logreg.score(X_train, y_train)))
print("Test set score: {:.3f}".format(logreg.score(X_test, y_test)))

# To view the coefficients as odds ratios we need to exponentiate them
coef_oddsratios = np.exp(logreg.coef_)
display(pd.DataFrame({
    "feature": X.columns,
    "odds_ratio": coef_oddsratios[0]
}))

# a confusion matrix compares the actual test data target values with what the model
# the results form with matrix of true positive, false positive on top
# and false negative, true negative positive on bottom
from sklearn.metrics import confusion_matrix
y_pred = logreg.predict(X_test)
print(confusion_matrix(y_test,y_pred))
```

Training set score: 0.726
Test set score: 0.727

	feature	odds_ratio
0	total_population	0.999921
1	poverty_rate	1.009632
2	no_vehicle_rate	1.000767
3	Open_Space	0.917465
4	Residential	1.029972

```
[[ 0 52 0]
 [ 0 280 0]
 [ 0 53 0]]
```

In []: